

Python Basic + Linux Intro

▼ History

Who created it?

Python was created by **Guido van Rossum**, a Dutch programmer. the first version was released in **1991**.

Why was Python made?

Guido wanted a language that was:

- Easy to read and write
- Powerful but beginner-friendly
- More productive than older languages like C or Java

How did it grow?

Over the years, Python became popular in:

- Web development
- Data science and AI
- Education
- Automation and scripting

Today

Python is one of the **most popular languages in the world**, used by companies like Google, Netflix, NASA, and even in high school CS classes!

Why 🍊 Python: simplicity + power.

▼ Syntax

▼ Variable Declaration Comparison

```
# Python
user_age = 25
user_name = "Alice"
```

```
// Java
int userAge = 25;
String userName = "Alice";
```

```
// JavaScript
const userAge = 25;
let userName = "Alice";
```

```
// C#
int userAge = 25;
string userName = "Alice";
```

```
// C++
int userAge = 25;
std::string
userName = "Alice";
```

▼ Variables and data types

1. Declaring Variables: saving data

Variable Assignment: Use = to assign values to variables.

```
score = 750
```

- score is the variable name, and 750 is its value.

2. Dynamically typed (no need to declare type)

```
x = 5      # int
name = "Ana" # str
pi = 3.14   # float
is_valid = True # bool
```

3. **Printing Variables:** Displays the value of a variable.

```
print(score)
```

- Result: Displays **750**.



Print command : Printing to the Screen

Printing String Data Type: Text enclosed in quotes.

```
print("Hello world!")
print('Hello world!')
```

- Double quotes, Single quotes. Both print the same output.

Printing Different Data Types:

- Numbers: Can print numbers directly.
- Variables

```
print(42)
print(score)
```

- Mixture (👉Practice)

4. **Operations and Expressions**

Arithmetic Operators

- **Addition:** +
- **Subtraction:** -
- **Multiplication:** *
- **Division:** /
- **Modulus:** %

Comparison Operators

- **Equal to:** ==
- **Not equal to:** !=
- **Greater than:** >
- **Less than:** <
- **Greater than or equal to:** >=
- **Less than or equal to:** <=

Logical Operators

- **AND:** and
- **OR:** or
- **NOT:** not

(👉practice) Figure out Boolean logic Table

```
# Logical Operators
x = True
y = False
```

▼ Control Flow and Basic Syntax

Basic rule of code execution

1. **Ignoring Empty Lines:** Python ignores blank lines in code.

```
...
...
print(score)
```

2. **Indentation matters:** Blocks of code use *spaces* or *tabs* (no braces `{ }` like in C or Java)

3. **Comments:**

```
# This is a comment
```

Control Flow

1. **Conditional Statement:** if / elif / else

- **Purpose: allow for dynamic decision-making in code based on conditions.**
- **Structure:**
 - Starts with if
 - Followed by a condition
 - Ends with a colon :
 - Indented code block runs if the condition is True

```
having_fun = "yes"
if having_fun == "yes":
    print(Glad you're having fun)
```

(practice 1) Change variable and try elif / else

(practice 2)

1. Write a program that asks the user for their test score (0-100).
(hint: Define a variable score with a value between 0 and 100.)
2. Print the grade based on the following criteria:

2. **while** and **for** loops (with `range()` often)

- **For Loop: Used to iterate over a sequence (like a list, tuple, string, or range).**

```
for item in sequence:
    # Code to execute
```

- **While Loop: Repeats as long as a condition is True, useful for indefinite iteration.**

```
while condition:
    # Code to execute
```

3. **Data Structures**

- **List:** `my_list = [1, 2, 3]`
Ordered, Changeable.

Use it when you want a collection of items in order and want to change them later

- **Tuple:** `my_tuple = (1, 2)`
Ordered,
Unchangeable
 - Like a list, but **you can't change the values after creation**
 - **Useful for fixed data** like coordinates or dates
 - **Faster than lists** in some cases
- **Set:** `my_set = {1, 2, 3}`
Unordered, No Duplicates
You **can't access items by index**. Good for checking if something exists quickly
- **Dictionary:** `my_dict = {"name": "Ana", "age": 16}`
Use when each item has a **name (key)** and a **value**. **Fast look-up by key**.
Ordered (since Python 3.7)

(👉 practice) **For** loop and **While** loop

▼ Functions and Modules

DRY Principle - Don't Repeat Yourself.

- **Definition:** A block of code grouped together with a name.
- **Purpose:** Write simple, easy-to-read, and non-repetitive code.

1. Creating a Function:

- **Keyword:** `def`
- **Name:** `say_hello`

- **Parameters: (name)**
 - **Colon: :**
 - **Body:** Indented code block
2. **Calling a Function: Function name followed by parentheses, including any required inputs.**
 3. Modules

Python Standard Library (*built-in*):

- **A set of modules included with every Python installation.**
- You **don't need to import anything** to use them. They are **available by default**.
- `print()` , `input()` , `len()` , `type()` , `int()` , `str()` , `sum()` , etc.

Function	Purpose
<code>print()</code>	Displays text or data to the screen
<code>input()</code>	Gets user input as a string
<code>len()</code>	Returns the length of a collection (like a list or string)
<code>type()</code>	Tells you the type of a variable
<code>int()</code>	Converts data to an integer
<code>str()</code>	Converts data to a string
<code>sum()</code>	Adds up numbers in a list or tuple

Importing Other Module:

- Syntax: `import module_name`

```
import math
print(math.sqrt(16)) # 4.0
```

```
import random
# Create a list of numbers
numbers = [1, 2, 3, 4, 5]
# Shuffle the list and print
```

```
random.shuffle(numbers)
print(numbers)
```

▼ Class and Object

Class

A **class** is a **blueprint** for creating objects.

Think of it like a recipe or a design — it doesn't *do* anything on its own until you *use* it to create something.

```
class Car:
    def __init__(self, brand, year):
        self.brand = brand
        self.year = year

    def start_engine(self):
        print(f"{self.brand} engine started!")
```

Term	Meaning	Example
<code>class</code>	Defines a blueprint	<code>class Dog:</code>
<code>__init__</code>	Special method that runs when object is created	like a setup function
<code>self</code>	Refers to the object itself	Always the first parameter
Attributes	Variables stored in the object	<code>self.brand = brand</code>
Methods	Functions inside the class	<code>def start_engine(self):</code>
Object	Instance of a class	<code>car1 = Car("Honda", 2022)</code>

Object

An **object** is an **actual thing** created from a class — like a specific car built from the blueprint.

```
my_car = Car("Mini Cooper S", 2020) # ← Object created
my_car.start_engine()      # Output: Mini Cooper S engine started!
```

(👉practice) Create a class called `Book` with the following:

- Attributes: `title`, `author`, and `year`
- Method: `describe()` that prints something like:
`"Title: Harry Potter, Author: J.K. Rowling, Year: 1997"`
- Add a method `is_old()` that returns `True` if the book is more than 20 years old.